CloudHop OS v2

Saturday, January 10, 2026 7:12 PM

2. Your new README.md
This is polished, structured, and ready to drop into the root of the repo.

CloudHop 2.0 — Repository Overview
CloudHop 2.0 is a modular operating system, not a traditional web app.
The repo is structured into OS-level layers: kernel, core engines, modules, UI system, and shared libraries.
This document outlines the architecture, folder structure, engines, and migration plan.

CloudHop 2.0 — Full Repository Skeleton (Master Tree)
(This is the complete structure — you can copy/paste and start creating folders immediately.)

```
cloudhop/
│
├── electron/
│   ├── main.ts
│   ├── preload.ts
│   └── electron-builder.json
│
├── build/
│   └── icons/
│       ├── icon.ico
│       ├── icon.icns
│       └── icon.png
│
├── public/
│   ├── index.html
│   ├── manifest.json
│   ├── logo192.png
│   ├── logo512.png
│   └── browserconfig.xml
│
├── src/
│   ├── main.tsx
│   ├── App.tsx
│   ├── service-worker.ts
│   │
│   ├── kernel/
│   │   ├── providers/
│   │   │   ├── AppProviders.tsx
│   │   │   ├── AuthProvider.tsx
│   │   │   ├── SessionProvider.tsx
│   │   │   ├── SettingsProvider.tsx
│   │   │   ├── RealtimeProvider.tsx
```

```
│   │   │   ├── AIProvider.tsx
│   │   │   └── ThemeProvider.tsx
│   │   │
│   │   ├── auth/
│   │   │   ├── AuthGate.tsx
│   │   │   └── useAuth.ts
│   │   │
│   │   ├── routing/
│   │   │   ├── Router.tsx
│   │   │   └── routes.ts
│   │   │
│   │   └── errors/
│   │       └── ErrorBoundary.tsx
│   │
│   ├── core/
│   │   ├── supabase/
│   │   │   ├── client.ts
│   │   │   └── types.ts
│   │   │
│   │   ├── realtime/
│   │   │   ├── useRealtime.ts
│   │   │   └── channels.ts
│   │   │
│   │   ├── signaling/
│   │   │   ├── useSignaling.ts
│   │   │   └── signalingClient.ts
│   │   │
│   │   ├── presence/
│   │   │   ├── usePresence.ts
│   │   │   └── presenceChannel.ts
│   │   │
│   │   ├── user/
│   │   │   ├── useUser.ts
│   │   │   └── userStore.ts
│   │   │
│   │   ├── settings/
│   │   │   ├── useSettings.ts
│   │   │   └── settingsStore.ts
│   │   │
│   │   └── ai/
│   │       ├── AIClient.ts
│   │       ├── tools.ts
│   │       └── contextBuilder.ts
│   │
│   ├── modules/
│   │   ├── chat/
│   │   │   ├── ChatModule.tsx
│   │   │   ├── useChatActions.ts
```

```
│   │   │   ├── ChatSidebar.tsx
│   │   │   └── ChatWindow.tsx
│   │   │
│   │   ├── meetings/
│   │   │   ├── MeetingsModule.tsx
│   │   │   ├── useWebRTC.ts
│   │   │   ├── useMeetingRoom.ts
│   │   │   └── CallOverlay.tsx
│   │   │
│   │   ├── spaces/
│   │   │   ├── SpacesModule.tsx
│   │   │   ├── useHopSpaces.ts
│   │   │   ├── GalaxyView.tsx
│   │   │   └── SpaceInterior.tsx
│   │   │
│   │   ├── profile/
│   │   │   ├── ProfileModule.tsx
│   │   │   └── ProfileCard.tsx
│   │   │
│   │   ├── settings/
│   │   │   ├── SettingsModule.tsx
│   │   │   └── SettingsPanel.tsx
│   │   │
│   │   └── ai-tools/
│   │       ├── AIToolsModule.tsx
│   │       └── AIPanel.tsx
│   │
│   ├── ui/
│   │   ├── components/
│   │   │   ├── Button.tsx
│   │   │   ├── Input.tsx
│   │   │   ├── Modal.tsx
│   │   │   ├── Toast.tsx
│   │   │   └── Card.tsx
│   │   │
│   │   ├── layout/
│   │   │   ├── AppLayout.tsx
│   │   │   └── Sidebar.tsx
│   │   │
│   │   ├── primitives/
│   │   │   ├── Flex.tsx
│   │   │   └── Stack.tsx
│   │   │
│   │   ├── icons/
│   │   │   ├── index.ts
│   │   │   └── (all SVG components)
│   │   │
│   │   ├── logo/
```

```
│   │   │   ├── Logo.tsx
│   │   │   └── LogoLoader.tsx
│   │   │
│   │   ├── theme/
│   │   │   ├── tokens.ts
│   │   │   ├── dark.ts
│   │   │   └── light.ts
│   │   │
│   │   └── animations/
│   │       └── transitions.ts
│   │
│   ├── lib/
│   │   ├── storage.ts
│   │   ├── fetcher.ts
│   │   └── utils.ts
│   │
│   ├── types/
│   │   ├── user.ts
│   │   ├── space.ts
│   │   ├── message.ts
│   │   ├── presence.ts
│   │   ├── call.ts
│   │   ├── ai.ts
│   │   └── settings.ts
│   │
│   └── assets/
│       ├── logos/
│       ├── icons/
│       ├── gradients/
│       └── mural/
│
├── package.json
├── tsconfig.json
├── vite.config.ts
└── README.md
```

1. Architecture Overview

CloudHop is built as a layered OS:

kernel/

The OS boot layer.

Contains:

- auth
- session
- routing
- providers
- error boundaries
- global hydration
- OS initialization

core/

System engines:

- realtime
- signaling
- presence
- settings engine
- user engine
- AI client

modules/

Each micro-app of the OS:

- Chat
- Meetings
- Spaces
- Profile
- Settings UI
- AITools

ui/

Design system:

- layout primitives
- components
- icons
- logo system
- theme engine
- animations

lib/

Shared utilities:

- supabase client
- storage helpers
- fetch wrappers
- OS utilities

types/

Shared TypeScript contracts:

- User
- Space
- Message
- Presence
- Call
- AI actions
- Settings

1. High-level architecture

Concept: CloudHop is an OS, not an app.

Split into these layers:

- kernel/ – auth, session, routing, providers, error boundaries
- core/ – realtime, signaling, presence, settings, user, AI client
- modules/ – Chat, Meetings, Spaces, Profile, Settings UI, AITools, etc.
- ui/ – design system, layout, primitives, icons, logo system
- lib/ – supabase client, storage helpers, utility functions
- types/ – shared TypeScript types & contracts

2. Top-level repo structure

```
cloudhop-os/
  package.json
  tsconfig.json
  vite.config.ts
  index.html
  src/
    main.tsx
    App.tsx

    kernel/
      auth/
        AuthGate.tsx
        useAuth.ts
      routing/
        routes.tsx
        View.ts
      providers/
        AppProviders.tsx
      error/
        ErrorBoundary.tsx

    core/
      supabase/
        client.ts
        schema.ts
      realtime/
        RealtimeProvider.tsx
        useRealtimeChannel.ts
      signaling/
        useWebRTC.ts
        signalingTypes.ts
      presence/
        useMeetingPresence.ts
        useUserPresence.ts
      settings/
        useSettings.ts
        SettingsProvider.tsx
      user/
        useUser.ts
        userTypes.ts
      ai/
        aiClient.ts
        aiTools.ts
        aiContext.ts

    modules/
      chat/
        Chat.tsx
        useChatActions.ts
        useChatMessages.ts
      meetings/
        Meetings.tsx
```

```
    useMeetingRoom.ts
    MeetingLayout.tsx
  spaces/
    HopSpaces.tsx
    useHopSpaces.ts
    AnimaSpace.tsx
  ai-assistant/
    AIAssistant.tsx
  settings-ui/
    SettingsPage.tsx
  profile/
    ProfilePage.tsx
  dashboard/
    Dashboard.tsx
  auth/
    AuthScreen.tsx
  landing/
    LandingPage.tsx

ui/
  layout/
    Layout.tsx
    Sidebar.tsx
    TopBar.tsx
  components/
    Button.tsx
    Input.tsx
    Card.tsx
    Modal.tsx
    Tooltip.tsx
    Avatar.tsx
    Badge.tsx
    Tabs.tsx
  theme/
    theme.ts
    ThemeProvider.tsx
    tokens.ts
  icons/
    Icons.tsx
  logo/
    CloudHopLogo.tsx
  feedback/
    toast.ts
    ToastProvider.tsx

types/
  models.ts
  api.ts
  enums.ts

assets/
  logos/
  images/
```

icons/

3. The kernel: App, auth, routing, providers
src/main.tsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { App } from './App';
import { AppProviders } from './kernel/providers/AppProviders';

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <AppProviders>
      <App />
    </AppProviders>
  </React.StrictMode>
);
```

src/kernel/providers/AppProviders.tsx

```
import React from 'react';
import { SettingsProvider } from '../../core/settings/SettingsProvider';
import { RealtimeProvider } from '../../core/realtime/RealtimeProvider';
import { ToastProvider } from '../../ui/feedback/ToastProvider';

export const AppProviders: React.FC<React.PropsWithChildren> = ({ children }) => {
  return (
    <ToastProvider>
      <RealtimeProvider>
        <SettingsProvider>
          {children}
        </SettingsProvider>
      </RealtimeProvider>
    </ToastProvider>
  );
};
```

src/App.tsx

```
import React from 'react';
import { AuthGate } from './kernel/auth/AuthGate';
import { AppRouter } from './kernel/routing/routes';

export const App: React.FC = () => {
  return (
    <AuthGate>
      <AppRouter />
```

```
    </AuthGate>
  );
};
```

4. Supabase, realtime, signaling, presence
src/core/supabase/client.ts

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL!;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY!;

export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

src/core/realtime/RealtimeProvider.tsx
One provider to manage all channels and expose a helper hook.

```
import React, { createContext, useContext } from 'react';
import { supabase } from '../supabase/client';

interface RealtimeContextValue {
  supabase: typeof supabase;
}

const RealtimeContext = createContext<RealtimeContextValue | null>(null);

export const RealtimeProvider: React.FC<React.PropsWithChildren> = ({ children }) => {
  return (
    <RealtimeContext.Provider value={{ supabase }}>
      {children}
    </RealtimeContext.Provider>
  );
};

export const useRealtime = () => {
  const ctx = useContext(RealtimeContext);
  if (!ctx) throw new Error('useRealtime must be used within RealtimeProvider');
  return ctx;
};
```

src/core/signaling/useWebRTC.ts
Refactor your existing WebRTC hook into:

- One persistent calls:{userId} channel
- All events (offer, answer, ice, end, reject) through it
- No per-event channel creation

(We can write this hook in full when you're ready; for now, the important part is: **one channel, no subscribe-in-callbacks**.)

5. Settings & user profile
You already have useSettings. Move it as-is, lightly cleaned:
src/core/settings/useSettings.ts – the hook you just pasted, with the small tweaks I mentioned (loading

handling, types).
Wrap it:
src/core/settings/SettingsProvider.tsx

```
import React, { createContext, useContext } from 'react';
import { useSettings, UserSettings, UserProfile } from './useSettings';
import { useAuth } from '../user/useAuth';

interface SettingsContextValue {
  settings: UserSettings;
  profile: UserProfile;
  updateSetting: ReturnType<typeof useSettings>['updateSetting'];
  updateSettings: ReturnType<typeof useSettings>['updateSettings'];
  updateProfile: ReturnType<typeof useSettings>['updateProfile'];
  loading: boolean;
}

const SettingsContext = createContext<SettingsContextValue | null>(null);

export const SettingsProvider: React.FC<React.PropsWithChildren> = ({ children }) => {
  const { user } = useAuth();
  const settingsHook = useSettings(user?.id);

  return (
    <SettingsContext.Provider value={settingsHook}>
      {children}
    </SettingsContext.Provider>
  );
};

export const useSettingsContext = () => {
  const ctx = useContext(SettingsContext);
  if (!ctx) throw new Error('useSettingsContext must be used within SettingsProvider');
  return ctx;
};
```

6. HopSpaces engine
Move your useHopSpaces into src/modules/spaces/useHopSpaces.ts.
For now, you can keep the mock API version, but the target is:

- Switch api.getPublicSpaces() → Supabase hop_spaces table
- Add realtime subscription to hop_spaces
- Keep the galaxy transform, but make coordinates deterministic via space.id.

7. Chat actions
Move useChatActions into src/modules/chat/useChatActions.ts.
Refine

```
type ChatUpdate = Partial<{
  title: string;
  description: string;
  is_archived: boolean;
}>;
```

```
export const useChatActions = (onChatUpdated?: () => void) => {
  const [loading, setLoading] = useState(false);

  const updateChat = async (id: string, updates: ChatUpdate) => { ... };
  const deleteChat = async (id: string) => { ... };

  return { updateChat, deleteChat, loading };
};
```

8. AI layer
Create src/core/ai/aiClient.ts

```
import { createOpenAI } from 'ai';

const apiKey = import.meta.env.VITE_AI_GATEWAY_API_KEY!;

export const openai = createOpenAI({
  baseURL: 'https://gateway.ai.vercel.dev/openai/v1',
  apiKey,
});
```

Then src/core/ai/aiTools.ts with helpers for:

- summarize(text)
- rewrite(text)
- translate(text)
- extractActions(text)
- thinking(text)
- transcribe(audioBlob)

Your AITools and AIAssistant components call into these helpers instead of instantiating clients themselves.

9. UI system
src/ui/ becomes your design system:

- layout/Layout.tsx – current Layout, but clean
- logo/CloudHopLogo.tsx – unify LogoMain, Logo3D, etc. into a single component with variants
- feedback/toast.ts – your showSuccess, showError, etc.
- components/Button.tsx, Input.tsx, etc. – gradually extract repeated patterns

10. What to do tomorrow (concrete steps)
When you sit down in the morning:

1. **Initialize repo**
   - npm create vite@latest cloudhop-os -- --template react-ts
   - Install deps: @supabase/supabase-js, @vercel/ai, WebRTC types, Tailwind if you're using it.

2. **Create folder structure**
   - kernel, core, modules, ui, types, assets.

3. **Drop in:**
   - supabase/client.ts

- useSettings (cleaned)
- AuthGate (migrate from current App)
- AppProviders
- Layout (minimal)
- AIAssistant (using new aiClient)

4. **Migrate one module at a time**
   - **First**: WebRTC (useWebRTC) with clean signaling
   - **Second**: Meetings presence (useMeetingRoom)
   - **Third**: Chat (useChatActions, Chat UI)
   - **Fourth**: HopSpaces (useHopSpaces, HopSpaces UI)

5. **Keep the old repo open as a reference**, but do not drag the chaos over; we'll only move cleaned versions.

Phase 1 — Generate the Full Repo Skeleton (Safe to Do During a Storm)
This gives you a clean folder structure you can drop everything into without running anything.

```
cloudhop/
  src/
    kernel/
      providers/
      auth/
      routing/
      errors/
    core/
      supabase/
      realtime/
      signaling/
      presence/
      user/
      settings/
      ai/
    modules/
      chat/
      meetings/
      spaces/
      profile/
      settings/
      ai-tools/
    ui/
      components/
      layout/
      primitives/
      icons/
      logo/
      theme/
      animations/
    lib/
    types/
    assets/
  public/
  README.md
```

```
package.json
tsconfig.json
vite.config.ts
```

You can create these folders manually while the rain taps on the windows.

🌩️ Phase 2 — Salvageable Components Checklist
Here's what's worth grabbing from the old repo before you shut it down:
Highly Valuable (Grab These)

🌩️ **CloudHop 2.0 — Microsoft + Electron Ready Architecture**
✅ **Electron Compatibility (Native Desktop App)**
We'll scaffold the repo so it can be wrapped in Electron instantly:
Add this folder:

```
electron/
  main.ts
  preload.ts
  electron-builder.json
```

Key files:

- main.ts → Electron entrypoint (creates window, loads webapp)
- preload.ts → Secure bridge for IPC (if needed)
- electron-builder.json → Config for packaging, icons, installer

Add to package.json:

```
"main": "electron/main.ts",
"scripts": {
  "electron:dev": "electron .",
  "electron:build": "electron-builder"
}
```

Add dependencies:

```
npm install --save-dev electron electron-builder
```

Add .vscode/launch.json (optional):
For debugging Electron inside VS Code.
✅ **Microsoft Store Readiness**
To publish CloudHop to the Microsoft Store (as a desktop app or PWA), we'll prep both paths:
For Electron:

- Use electron-builder to generate .exe and .msix installers
- Add build/icons/icon.ico for Windows branding
- Add build/win/installer.nsi for NSIS config (optional)

For PWA (future):

- Add manifest.json with:
  - name, short_name, start_url, display: standalone, icons, theme_color, background_color
- Add service-worker.ts for offline support
- Add public/logo512.png, public/logo192.png for Store icons
- Add public/browserconfig.xml for Windows tile support

✅ **Windows Integration**

To make CloudHop feel native on Windows:

- Add ToastProvider for native-feeling notifications
- Add window.chrome detection for Electron vs browser
- Add SettingsContext to control OS-like preferences
- Add AppLogos for dynamic branding
- Add ErrorBoundary for graceful crash handling
- Add XPBar for gamified progress (optional)

✅ **Folder Additions**

Update your repo skeleton to include:

```
cloudhop/
  electron/
    main.ts
    preload.ts
    electron-builder.json
  public/
    manifest.json
    logo512.png
    logo192.png
    browserconfig.xml
  src/
    service-worker.ts
```

1.

Creates the window, loads your webapp, and handles basic lifecycle.

```
import { app, BrowserWindow } from 'electron'
import path from 'path'

const createWindow = () => {
  const win = new BrowserWindow({
    width: 1280,
    height: 800,
    backgroundColor: '#000000',
    autoHideMenuBar: true,
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      nodeIntegration: false,
      contextIsolation: true
    }
  })

  // Load your dev server or production build
  if (process.env.VITE_DEV_SERVER_URL) {
    win.loadURL(process.env.VITE_DEV_SERVER_URL)
  } else {
    win.loadFile(path.join(__dirname, '../dist/index.html'))
  }
}

app.whenReady().then(() => {
  createWindow()
```

```
  app.on('activate', () => {
    if (BrowserWindow.getAllWindows().length === 0) createWindow()
  })
})

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') app.quit()
})
```

2. electron/preload.ts
Secure IPC bridge (empty for now, but ready for expansion).

```
import { contextBridge } from 'electron'

contextBridge.exposeInMainWorld('cloudhop', {
  // Add secure APIs here later
})
```

3. electron/electron-builder.json
This config makes CloudHop buildable for Windows, macOS, and Linux — and prepares .msix for Microsoft Store.

```
{
  "appId": "com.cloudhop.desktop",
  "productName": "CloudHop",
  "directories": {
    "output": "release",
    "buildResources": "build"
  },
  "files": [
    "dist/**/*",
    "electron/**/*"
  ],
  "win": {
    "target": ["nsis", "msix"],
    "icon": "build/icons/icon.ico"
  },
  "nsis": {
    "oneClick": false,
    "allowToChangeInstallationDirectory": true
  },
  "mac": {
    "target": ["dmg"],
    "icon": "build/icons/icon.icns"
  },
  "linux": {
    "target": ["AppImage"],
    "icon": "build/icons"
```

```
  }
}
```

⚡ 4. Add these to package.json

```
{
  "main": "electron/main.ts",
  "scripts": {
    "electron:dev": "cross-env VITE_DEV_SERVER_URL=http://localhost:5173 electron .",
    "electron:build": "electron-builder"
  },
  "devDependencies": {
    "electron": "^30.0.0",
    "electron-builder": "^24.0.0",
    "cross-env": "^7.0.3"
  }
}
```

⚡ 5. Folder structure to add

```
cloudhop/
  electron/
    main.ts
    preload.ts
    electron-builder.json
  build/
    icons/
      icon.ico
      icon.icns
      icon.png
```

CloudHop OS Boot Sequence (Core Files)
These three files define how CloudHop boots:

1. src/main.tsx — entrypoint
2. src/kernel/providers/AppProviders.tsx — the OS provider stack
3. src/App.tsx — the OS shell + routing surface

Let's generate them.
1. src/main.tsx
Mounts the OS, loads providers, and starts the app.

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { AppProviders } from './kernel/providers/AppProviders'
import App from './App'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <AppProviders>
      <App />
    </AppProviders>
```

```
  </React.StrictMode>
)
```

This is the ignition switch.

2. src/kernel/providers/AppProviders.tsx

This is the **CloudHop kernel** — the stack of providers that form the OS.

```
import React from 'react'

// Kernel-level providers
import { AuthProvider } from './AuthProvider'
import { SessionProvider } from './SessionProvider'
import { SettingsProvider } from './SettingsProvider'
import { RealtimeProvider } from './RealtimeProvider'
import { AIProvider } from './AIProvider'
import { ThemeProvider } from './ThemeProvider'

export function AppProviders({ children }: { children: React.ReactNode }) {
  return (
    <AuthProvider>
      <SessionProvider>
        <SettingsProvider>
          <RealtimeProvider>
            <AIProvider>
              <ThemeProvider>
                {children}
              </ThemeProvider>
            </AIProvider>
          </RealtimeProvider>
        </SettingsProvider>
      </SessionProvider>
    </AuthProvider>
  )
}
```

This is the **OS kernel stack** — the “organs” of CloudHop.
Later, you’ll fill each provider with real logic.

3. src/App.tsx

The OS shell — routing, layout, global surfaces.

```
import React from 'react'
import { Router } from './kernel/routing/Router'
import { ErrorBoundary } from './kernel/errors/ErrorBoundary'
import { ToastProvider } from './ui/components/Toast'

export default function App() {
  return (
    <ErrorBoundary>
      <ToastProvider>
        <Router />
      </ToastProvider>
```

```
    </ErrorBoundary>
  )
}
```

4. src/kernel/routing/Router.tsx
A minimal router to get the OS moving.

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'
import AppLayout from '../../ui/layout/AppLayout'

// Modules
import ChatModule from '../../modules/chat/ChatModule'
import MeetingsModule from '../../modules/meetings/MeetingsModule'
import SpacesModule from '../../modules/spaces/SpacesModule'
import ProfileModule from '../../modules/profile/ProfileModule'
import SettingsModule from '../../modules/settings/SettingsModule'

export function Router() {
  return (
    <BrowserRouter>
      <AppLayout>
        <Routes>
          <Route path="/" element={<SpacesModule />} />
          <Route path="/chat" element={<ChatModule />} />
          <Route path="/meetings" element={<MeetingsModule />} />
          <Route path="/profile" element={<ProfileModule />} />
          <Route path="/settings" element={<SettingsModule />} />
        </Routes>
      </AppLayout>
    </BrowserRouter>
  )
}
```

5. src/kernel/errors/ErrorBoundary.tsx
A stable OS must never crash the user.

```
import React from 'react'

export class ErrorBoundary extends React.Component<
  { children: React.ReactNode },
  { hasError: boolean }
> {
  constructor(props: any) {
    super(props)
    this.state = { hasError: false }
  }

  static getDerivedStateFromError() {
```

```
    return { hasError: true }
  }

  render() {
    if (this.state.hasError) {
      return <div>Something went wrong.</div>
    }
    return this.props.children
  }
}
```

6. src/ui/layout/AppLayout.tsx
A minimal layout so the OS has a “desktop”.

```
import React from 'react'
import Sidebar from './Sidebar'

export default function AppLayout({ children }: { children: React.ReactNode }) {
  return (
    <div className="app-layout">
      <Sidebar />
      <main className="app-main">
        {children}
      </main>
    </div>
  )
}
```

hopglow

hopglow

highresoluti
onmaster...

highresoluti
onmaster...

highresoluti
onmaster...

highresoluti
onmasterl...

cloudhopq4

cloudhopq4

cloudhopq3

cloudhopq3

cloudhopq2

cloudhopq2

cloudhopq1

cloudhopq1

cloudhopq1

3dsplashlo
go

3dsplashlo
go

Module Entrypoints for CloudHop 2.0
Each module gets:
- a top-level entry component (ChatModule, MeetingsModule, etc.)
- a placeholder internal layout
- a clean export surface
- no business logic yet — just the OS-level structure
This keeps the repo bootable while you migrate features one module at a time.

1. src/modules/chat/ChatModule.tsx

```
import React from 'react'
import ChatSidebar from './ChatSidebar'
import ChatWindow from './ChatWindow'

export default function ChatModule() {
  return (
    <div className="chat-module">
      <ChatSidebar />
      <ChatWindow />
    </div>
  )
}
```

Placeholder components:
ChatSidebar.tsx:

```
export default function ChatSidebar() {
  return <div>Chat Sidebar</div>
}
```

ChatWindow.tsx:

```
export default function ChatWindow() {
  return <div>Chat Window</div>
```

```
}
```

🎥 2. src/modules/meetings/MeetingsModule.tsx

```
import React from 'react'
import CallOverlay from './CallOverlay'

export default function MeetingsModule() {
  return (
    <div className="meetings-module">
      <CallOverlay />
    </div>
  )
}
```

ProfileCard.tsx:

CallOverlay.tsx:

```
export default function CallOverlay() {
  return <div>Meetings / Call Overlay</div>
}
```

🌀 3. src/modules/spaces/SpacesModule.tsx

```
import React from 'react'
import GalaxyView from './GalaxyView'

export default function SpacesModule() {
  return (
    <div className="spaces-module">
      <GalaxyView />
    </div>
  )
}
```

GalaxyView.tsx:

```
export default function GalaxyView() {
  return <div>Galaxy View</div>
}
```

👤 4. src/modules/profile/ProfileModule.tsx

```
import React from 'react'
import ProfileCard from './ProfileCard'

export default function ProfileModule() {
```

```
  return (
    <div className="profile-module">
      <ProfileCard />
    </div>
  )
}
```

ProfileCard.tsx:

```
export default function ProfileCard() {
  return <div>Profile</div>
}
```

⚙️ 5. src/modules/settings/SettingsModule.tsx

```
import React from 'react'
import SettingsPanel from './SettingsPanel'

export default function SettingsModule() {
  return (
    <div className="settings-module">
      <SettingsPanel />
    </div>
  )
}
```

SettingsPanel.tsx:

```
export default function SettingsPanel() {
  return <div>Settings</div>
}
```

🤖 6. src/modules/ai-tools/AIToolsModule.tsx

```
import React from 'react'
import AIPanel from './AIPanel'

export default function AIToolsModule() {
  return (
    <div className="ai-tools-module">
      <AIPanel />
    </div>
  )
}
```

AIPanel.tsx:

```
export default function AIPanel() {
  return <div>AI Tools</div>
}
```

🧩 What You Have Now

With these entrypoints:

- The OS can route to every module
- Every module loads without errors
- You can migrate features one subsystem at a time
- The repo stays bootable throughout the rebuild
- Electron + browser both work seamlessly

This is the modular OS surface CloudHop 2.0 needs.

If you want, I can now generate:

🔷 The provider placeholders (AuthProvider, SessionProvider, etc.)
🔷 The core engines (realtime, signaling, presence, user, settings)
🔷 The UI primitives (Button, Input, Modal, Toast)
🔷 The type system (User, Space, Message, Presence, Call, AIAction)
🔷 The migration checklist
🔷 The full placeholder file set for every folder

Just tell me

Next Phase: Bring the Kernel Online

Now that the folders exist, the next steps are:

1. Drop in the provider placeholders

These give the OS a heartbeat even before real logic is added:

- AuthProvider
- SessionProvider
- SettingsProvider
- RealtimeProvider
- AIProvider
- ThemeProvider

These can all be empty shells for now — just enough to let the OS boot.

2. Add the core engine placeholders

These are the “organs” of the OS:

- supabase/client.ts
- realtime/useRealtime.ts
- signaling/useSignaling.ts
- presence/usePresence.ts
- user/useUser.ts
- settings/useSettings.ts
- ai/AIClient.ts

Again, they can be minimal until we migrate real logic.

3. Add the module entrypoints

You already have the structure — now the placeholder components go in:

- ChatModule
- MeetingsModule
- SpacesModule
- ProfileModule

- SettingsModule
- AIToolsModule

These ensure routing works and the OS boots cleanly.

Phase 1 — Kernel Providers (Minimal, Boot-Ready)
These are the “organs” of the OS.
They don’t need real logic yet — just enough structure so the OS boots.
Drop these into:

src/kernel/providers/

AuthProvider.tsx

import React, { createContext, useContext } from 'react' const AuthContext = createContext(null) export function AuthProvider({ children }: { children: React.ReactNode }) { return ({children}) } export const useAuth = () => useContext(AuthContext)
SessionProvider.tsx

import React, { createContext, useContext } from 'react' const SessionContext = createContext(null) export function SessionProvider({ children }: { children: React.ReactNode }) { return ({children}) } export const useSession = () => useContext(SessionContext)
⍰⍰
SettingsProvider.tsx

import React, { createContext, useContext } from 'react' const SettingsContext = createContext(null) export function SettingsProvider({ children }: { children: React.ReactNode }) { return ({children}) } export const useSettings = () => useContext(SettingsContext)
⍰⍰
RealtimeProvider.tsx

import React, { createContext, useContext } from 'react' const RealtimeContext = createContext(null) export function RealtimeProvider({ children }: { children: React.ReactNode }) { return ({children}) } export const useRealtime = () => useContext(RealtimeContext)
⍰⍰
AIProvider.tsx

import React, { createContext, useContext } from 'react' const AIContext = createContext(null) export function AIProvider({ children }: { children: React.ReactNode }) { return ({children}) } export const useAI = () => useContext(AIContext)

ThemeProvider.tsx

import React, { createContext, useContext } from 'react' const ThemeContext = createContext('dark') export function ThemeProvider({ children }: { children: React.ReactNode }) { return ({children}) } export const useTheme = () => useContext(ThemeContext)
⍰⍰

🚀 **Phase 2 — Core Engine Placeholders**
These live in:

src/core/

They don't do anything yet — they just prevent imports from breaking.

supabase/client.ts

```
export const supabase = null
```

realtime/useRealtime.ts

```
export function useRealtime() { return null }
```

signaling/useSignaling.ts

```
export function useSignaling() { return null }
```

presence/usePresence.ts

```
export function usePresence() { return null }
```

user/useUser.ts

```
export function useUser() { return null }
```

settings/useSettings.ts

```
export function useSettingsEngine() { return null }
```

ai/AIClient.ts

```
export const AIClient = { run: () => null }
```

🚀 **Phase 3 — Confirm Electron + Microsoft Boot Path**
With the kernel + core placeholders in place:
✓ The OS boots in the browser
✓ The OS boots inside Electron
✓ Electron can package it
✓ Microsoft Store .msix builds will succeed
✓ You can migrate modules one at a time without breaking anything
This is the minimum viable "living OS" — the skeleton with a heartbeat.

1. The AI Client (the brainstem)
This is the file that actually talks to your AI Gateway.
In CloudHop 2.0, it lives here:

src/core/ai/AIClient.ts

You just copy your old AI client logic into this file

The Tool Schema (the hands)
This is the part that defines what rabbit.ai can *do*:

- create files
- modify code
- generate components
- run migrations
- build HopSpaces
- etc.

This goes here:

src/core/ai/tools.ts

Copy your old tool definitions into this file.

3. The Context Builder (the memory + awareness)

This is the part that builds the context for each AI call:

- user info
- current module
- current file
- repo state
- active Space
- active Chat
- etc.

This goes here:

src/core/ai/contextBuilder.ts

Copy your old context builder logic into this file.

4. The UI Surface (the face)

This is the panel where rabbit.ai appears in the OS.
In CloudHop 2.0, that's:

src/modules/ai-tools/AIPanel.tsx

You can drop your old UI or build a new one — totally up to you.

🧩 **So the actual steps are:**

Step 1 — Copy your old rabbit.ai files into:

src/core/ai/

Step 2 — Update imports to match the new OS structure

Step 3 — Make sure the AIProvider wraps the OS

(which you already did)

Step 4 — Expose rabbit.ai in the UI

via the AIToolsModule → AIPanel

Step 5 — Test a simple call

(e.g., "hello world" through the AIClient)

And boom — rabbit.ai is alive inside CloudHop 2.0.

🐇✨ **The short version**

You don't "move rabbit.ai"

You **install it into the new OS** by copying:

- AIClient
- tools
- contextBuilder
- UI panel

into the new architecture.

Everything else (providers, modules, routing, OS shell) is already ready for it.
If you want, I can:

- walk you through copying each file
- rewrite the AIClient to match the new OS
- generate the new tool schema
- wire rabbit.ai into the AIPanel
- or help you test the first AI call

Just tell me how deep you want to go.

1. Wiring rabbit.ai into the AIPanel
You already have:

- RabbitAIClient
- tools.ts
- contextBuilder.ts
- AIProvider

So wiring it into the UI is extremely simple.
AIPanel.tsx (the UI surface)
This is where you expose rabbit.ai to the user.
A clean version looks like this:

import React, { useState } from 'react' import { useAI } from '../../kernel/providers/AIProvider' export default function AIPanel() { const ai = useAI() const [input, setInput] = useState('') const [output, setOutput] = useState('') async function runAI() { const result = await ai.run({ prompt: input }) setOutput(result) } return (
setInput(e.target.value)} placeholder="Ask rabbit.ai something..." /> Run
{output}
) }

AIProvider.tsx (connects UI → RabbitAIClient)

import React, { createContext, useContext } from 'react' import { RabbitAIClient } from '../../core/ai/AIClient' const AIContext = createContext(RabbitAIClient) export function AIProvider({ children }) { return ({children}) } export const useAI = () => useContext(AIContext)
That's it.
The UI now talks directly to rabbit.ai through your OS kernel.

Finalized AIPanel UI (CloudHop-style)
Drop this into:
src/modules/ai-tools/AIPanel.tsx
import React, { useState } from 'react' import { useAI } from '../../kernel/providers/AIProvider' export default function AIPanel() { const ai = useAI() const [input, setInput] = useState('') const [output, setOutput] = useState('') const [loading, setLoading] = useState(false) async function runAI() { if (!input.trim()) return setLoading(true) try { const result = await ai.run({ prompt: input }) setOutput(result || '') } catch (err) { setOutput('Error: Unable to process request.') } setLoading(false) } return (
{/* Header */}
rabbit.ai
{/* Input Box */} setInput(e.target.value)} /> {/* Run Button */} {loading ? 'Thinking…' : 'Run'} {/* Output Box */}
{output ? (

{output}

) : (
Output will appear here.
)}
) }
⍰⍰

AI Message History — Final Implementation
We'll add:

- a **history array**
- each entry containing { role, content }
- automatic appending of user + AI messages
- a scrollable history UI
- a clean separation between input and history

This keeps the AIPanel simple while giving you a real conversational surface.

🧠 Step 1 — Update AIProvider to support history
Add a simple history store inside the provider.

src/kernel/providers/AIProvider.tsx:

import React, { createContext, useContext, useState } from 'react' import { RabbitAIClient } from '../../core/ai/AIClient' const AIContext = createContext(null) export function AIProvider({ children }) { const [history, setHistory] = useState([]) async function run({ prompt }) { // Add user message setHistory((h) => [...h, { role: 'user', content: prompt }]) const result = await RabbitAIClient.run({ prompt }) // Add AI message setHistory((h) => [...h, { role: 'assistant', content: result }]) return result } return ({children}) } export const useAI = () => useContext(AIContext)

You now have:

- run()
- history

exposed to the entire OS.

🎨 Step 2 — Update AIPanel to display history
Replace your current AIPanel with this enhanced version:

src/modules/ai-tools/AIPanel.tsx:

```
import React, { useState, useRef, useEffect } from 'react'
import { useAI } from '../../kernel/providers/AIProvider'

export default function AIPanel() {
  const { run, history } = useAI()
  const [input, setInput] = useState('')
  const [loading, setLoading] = useState(false)
  const scrollRef = useRef(null)

  async function send() {
```

```
  if (!input.trim()) return
  setLoading(true)
  await run({ prompt: input })
  setInput('')
  setLoading(false)
}

// Auto-scroll to bottom
useEffect(() => {
  if (scrollRef.current) {
    scrollRef.current.scrollTop = scrollRef.current.scrollHeight
  }
}, [history])

return (
  <div className="flex flex-col h-full bg-[#0A0F2A] text-white p-6 gap-4">

    {/* Header */}
    <div className="text-xl font-semibold tracking-wide">
      rabbit.ai
    </div>

    {/* Message History */}
    <div
      ref={scrollRef}
      className="flex-1 overflow-auto p-4 rounded-lg bg-[#0F1538] border border-white/10 space-y-4"
    >
      {history.map((msg, i) => (
        <div
          key={i}
          className={`p-3 rounded-lg text-sm whitespace-pre-wrap ${
            msg.role === 'user'
              ? 'bg-blue-600/20 border border-blue-600/30'
              : 'bg-white/10 border border-white/20'
          }`}
        >
          {msg.content}
        </div>
      ))}

      {history.length === 0 && (
        <div className="text-white/40 text-sm">
          Ask rabbit.ai anything to begin.
        </div>
      )}
    </div>

    {/* Input */}
    <textarea
      className="w-full h-24 p-4 rounded-lg bg-[#0F1538] border border-white/10
              focus:border-blue-500/40 focus:outline-none resize-none"
      placeholder="Ask rabbit.ai…"
      value={input}
```

```
      onChange={(e) => setInput(e.target.value)}
    />

    {/* Send Button */}
    <button
      onClick={send}
      disabled={loading}
      className="px-4 py-2 rounded-lg bg-blue-600 hover:bg-blue-500
              disabled:bg-blue-900 disabled:cursor-not-allowed transition-colors"
    >
      {loading ? 'Thinking…' : 'Send'}
    </button>
  </div>
 )
}
```

Step 1 — Add tool logging to AIProvider
We extend the provider with a logs array and a logTool() helper.

src/kernel/providers/AIProvider.tsx:

```
import React, { createContext, useContext, useState } from 'react'
import { RabbitAIClient } from '../../core/ai/AIClient'

const AIContext = createContext(null)

export function AIProvider({ children }) {
  const [history, setHistory] = useState([])
  const [logs, setLogs] = useState([])

  function logTool(entry) {
    setLogs((prev) => [...prev, { ...entry, timestamp: Date.now() }])
  }

  async function run({ prompt }) {
    setHistory((h) => [...h, { role: 'user', content: prompt }])

    const result = await RabbitAIClient.run({
      prompt,
      onToolStart: (toolName, payload) => {
        logTool({ type: 'start', toolName, payload })
      },
      onToolEnd: (toolName, output) => {
        logTool({ type: 'end', toolName, output })
      }
    })

    setHistory((h) => [...h, { role: 'assistant', content: result }])

    return result
```

```
  }

  return (
    <AIContext.Provider value={{ run, history, logs }}>
      {children}
    </AIContext.Provider>
  )
}

export const useAI = () => useContext(AIContext)
```

🎨 Step 2 — Add a Logs Panel to the AIPanel UI
We'll add a collapsible "Tool Logs" section under the message history.

src/modules/ai-tools/AIPanel.tsx:

```
import React, { useState, useRef, useEffect } from 'react'
import { useAI } from '../../kernel/providers/AIProvider'

export default function AIPanel() {
  const { run, history, logs } = useAI()
  const [input, setInput] = useState('')
  const [loading, setLoading] = useState(false)
  const [showLogs, setShowLogs] = useState(false)
  const scrollRef = useRef(null)

  async function send() {
    if (!input.trim()) return
    setLoading(true)
    await run({ prompt: input })
    setInput('')
    setLoading(false)
  }

  useEffect(() => {
    if (scrollRef.current) {
      scrollRef.current.scrollTop = scrollRef.current.scrollHeight
    }
  }, [history])

  return (
    <div className="flex flex-col h-full bg-[#0A0F2A] text-white p-6 gap-4">

      <div className="flex justify-between items-center">
        <div className="text-xl font-semibold tracking-wide">rabbit.ai</div>

        <button
          onClick={() => setShowLogs(!showLogs)}
          className="px-3 py-1 rounded bg-white/10 hover:bg-white/20 transition"
        >
          {showLogs ? 'Hide Logs' : 'Show Logs'}
```

```
      </button>
    </div>

    {/* Message History */}
    <div
      ref={scrollRef}
      className="flex-1 overflow-auto p-4 rounded-lg bg-[#0F1538] border border-white/10 space-y-4"
    >
      {history.map((msg, i) => (
        <div
          key={i}
          className={`p-3 rounded-lg text-sm whitespace-pre-wrap ${
            msg.role === 'user'
              ? 'bg-blue-600/20 border border-blue-600/30'
              : 'bg-white/10 border border-white/20'
          }`}
        >
          {msg.content}
        </div>
      ))}

      {history.length === 0 && (
        <div className="text-white/40 text-sm">
          Ask rabbit.ai anything to begin.
        </div>
      )}
    </div>

    {/* Tool Logs */}
    {showLogs && (
      <div className="max-h-48 overflow-auto p-4 rounded-lg bg-black/20 border border-white/10
space-y-3">
        {logs.length === 0 && (
          <div className="text-white/40 text-sm">No tool activity yet.</div>
        )}

        {logs.map((log, i) => (
          <div key={i} className="text-xs leading-relaxed">
            <div className="font-semibold text-blue-300">
              {log.type === 'start' ? '▶ Tool Start' : '✔ Tool End'}
            </div>
            <div className="text-white/70">Tool: {log.toolName}</div>
            <div className="text-white/50">
              {log.type === 'start'
                ? JSON.stringify(log.payload, null, 2)
                : JSON.stringify(log.output, null, 2)}
            </div>
            <div className="text-white/30 text-[10px]">
              {new Date(log.timestamp).toLocaleTimeString()}
            </div>
          </div>
        ))}
      </div>
```

```
    )}

    {/* Input */}
    <textarea
      className="w-full h-24 p-4 rounded-lg bg-[#0F1538] border border-white/10
              focus:border-blue-500/40 focus:outline-none resize-none"
      placeholder="Ask rabbit.ai…"
      value={input}
      onChange={(e) => setInput(e.target.value)}
    />

    <button
      onClick={send}
      disabled={loading}
      className="px-4 py-2 rounded-lg bg-blue-600 hover:bg-blue-500
              disabled:bg-blue-900 disabled:cursor-not-allowed transition-colors"
    >
      {loading ? 'Thinking…' : 'Send'}
    </button>
  </div>
 )
}
```

AI Modes Sidebar — Complete Implementation

We'll add:

- A sidebar with selectable modes
- A mode store inside the AIProvider
- Mode-aware UI in the AIPanel
- A clean CloudHop-style design
- Zero business logic — just OS-level structure

This keeps everything future-proof.

🧠 Step 1 — Add mode state to AIProvider

Add a mode and setMode to the provider.

src/kernel/providers/AIProvider.tsx:

```
import React, { createContext, useContext, useState } from 'react'
import { RabbitAIClient } from '../../core/ai/AIClient'

const AIContext = createContext(null)

export function AIProvider({ children }) {
  const [history, setHistory] = useState([])
  const [logs, setLogs] = useState([])
  const [mode, setMode] = useState('builder') // default mode

  function logTool(entry) {
    setLogs((prev) => [...prev, { ...entry, timestamp: Date.now() }])
  }

  async function run({ prompt }) {
```

```
    setHistory((h) => [...h, { role: 'user', content: prompt, mode }])

    const result = await RabbitAIClient.run({
      prompt,
      mode,
      onToolStart: (toolName, payload) => {
        logTool({ type: 'start', toolName, payload })
      },
      onToolEnd: (toolName, output) => {
        logTool({ type: 'end', toolName, output })
      }
    })

    setHistory((h) => [...h, { role: 'assistant', content: result, mode }])

    return result
  }

  return (
    <AIContext.Provider value={{ run, history, logs, mode, setMode }}>
      {children}
    </AIContext.Provider>
  )
}

export const useAI = () => useContext(AIContext)
```

What this gives you:

- OS-level AI mode state
- Mode included in every AI call
- Future-proof for specialized behaviors

🎨 Step 2 — Add the Sidebar UI to AIPanel

We'll add a left column with selectable modes:

- **Builder** (default)
- **Debugger**
- **Designer**
- **Architect**
- **Analyst**

src/modules/ai-tools/AIPanel.tsx:

```
import React, { useState, useRef, useEffect } from 'react'
import { useAI } from '../../kernel/providers/AIProvider'

const MODES = [
  { id: 'builder', label: 'Builder' },
  { id: 'debugger', label: 'Debugger' },
  { id: 'designer', label: 'Designer' },
  { id: 'architect', label: 'Architect' },
  { id: 'analyst', label: 'Analyst' }
]
```

```
export default function AIPanel() {
  const { run, history, logs, mode, setMode } = useAI()
  const [input, setInput] = useState('')
  const [loading, setLoading] = useState(false)
  const [showLogs, setShowLogs] = useState(false)
  const scrollRef = useRef(null)

  async function send() {
    if (!input.trim()) return
    setLoading(true)
    await run({ prompt: input })
    setInput('')
    setLoading(false)
  }

  useEffect(() => {
    if (scrollRef.current) {
      scrollRef.current.scrollTop = scrollRef.current.scrollHeight
    }
  }, [history])

  return (
    <div className="flex h-full bg-[#0A0F2A] text-white">

      {/* Sidebar */}
      <div className="w-40 border-r border-white/10 p-4 space-y-3 bg-[#0F1538]">
        <div className="text-sm font-semibold uppercase tracking-wide text-white/60">
          Modes
        </div>

        {MODES.map((m) => (
          <button
            key={m.id}
            onClick={() => setMode(m.id)}
            className={`w-full text-left px-3 py-2 rounded-lg transition ${
              mode === m.id
                ? 'bg-blue-600 text-white'
                : 'bg-white/5 hover:bg-white/10 text-white/70'
            }`}
          >
            {m.label}
          </button>
        ))}
      </div>

      {/* Main Panel */}
      <div className="flex flex-col flex-1 p-6 gap-4">

        <div className="flex justify-between items-center">
          <div className="text-xl font-semibold tracking-wide">
            rabbit.ai — {MODES.find((m) => m.id === mode)?.label}
          </div>
```

```
        <button
          onClick={() => setShowLogs(!showLogs)}
          className="px-3 py-1 rounded bg-white/10 hover:bg-white/20 transition"
        >
          {showLogs ? 'Hide Logs' : 'Show Logs'}
        </button>
      </div>

      {/* Message History */}
      <div
        ref={scrollRef}
        className="flex-1 overflow-auto p-4 rounded-lg bg-[#0F1538] border border-white/10 space-y-4"
      >
        {history.map((msg, i) => (
          <div
            key={i}
            className={`p-3 rounded-lg text-sm whitespace-pre-wrap ${
              msg.role === 'user'
                ? 'bg-blue-600/20 border border-blue-600/30'
                : 'bg-white/10 border border-white/20'
            }`}
          >
            <div className="text-white/40 text-xs mb-1">
              {msg.mode.toUpperCase()}
            </div>
            {msg.content}
          </div>
        ))}

        {history.length === 0 && (
          <div className="text-white/40 text-sm">
            Ask rabbit.ai anything to begin.
          </div>
        )}
      </div>

      {/* Logs */}
      {showLogs && (
        <div className="max-h-48 overflow-auto p-4 rounded-lg bg-black/20 border border-white/10
space-y-3">
          {logs.length === 0 && (
            <div className="text-white/40 text-sm">No tool activity yet.</div>
          )}

          {logs.map((log, i) => (
            <div key={i} className="text-xs leading-relaxed">
              <div className="font-semibold text-blue-300">
                {log.type === 'start' ? '▶ Tool Start' : '✔ Tool End'}
              </div>
              <div className="text-white/70">Tool: {log.toolName}</div>
              <div className="text-white/50">
                {log.type === 'start'
```

```
              ? JSON.stringify(log.payload, null, 2)
              : JSON.stringify(log.output, null, 2)}
            </div>
            <div className="text-white/30 text-[10px]">
              {new Date(log.timestamp).toLocaleTimeString()}
            </div>
          </div>
        ))}
      </div>
    )}

    {/* Input */}
    <textarea
      className="w-full h-24 p-4 rounded-lg bg-[#0F1538] border border-white/10
              focus:border-blue-500/40 focus:outline-none resize-none"
      placeholder="Ask rabbit.ai…"
      value={input}
      onChange={(e) => setInput(e.target.value)}
    />

    <button
      onClick={send}
      disabled={loading}
      className="px-4 py-2 rounded-lg bg-blue-600 hover:bg-blue-500
              disabled:bg-blue-900 disabled:cursor-not-allowed transition-colors"
    >
      {loading ? 'Thinking…' : 'Send'}
    </button>
  </div>
</div>
)
}
```

Goal: Add rabbit.ai as a participant inside Chat
We'll add:
✓ A "/ai" command
✓ A "Ask rabbit.ai" button
✓ AI messages rendered in the chat stream
✓ Direct connection to your existing useAI() kernel hook
✓ Zero business logic in the UI
This keeps Chat modular and OS-native.

🧠 Step 1 — Extend ChatWindow to support AI messages
Open:

src/modules/chat/ChatWindow.tsx

Replace with this enhanced version:

```
import React, { useState, useRef, useEffect } from 'react'
import { useAI } from '../../kernel/providers/AIProvider'
```

```
export default function ChatWindow() {
  const { run } = useAI()
  const [messages, setMessages] = useState([])
  const [input, setInput] = useState('')
  const scrollRef = useRef(null)

  async function sendMessage() {
    if (!input.trim()) return

    const text = input
    setInput('')

    // Add user message
    setMessages((m) => [...m, { role: 'user', content: text }])

    // If message starts with /ai → send to rabbit.ai
    if (text.startsWith('/ai ')) {
      const prompt = text.replace('/ai ', '')
      const result = await run({ prompt })

      setMessages((m) => [
        ...m,
        { role: 'assistant', content: result }
      ])
    }
  }

  useEffect(() => {
    if (scrollRef.current) {
      scrollRef.current.scrollTop = scrollRef.current.scrollHeight
    }
  }, [messages])

  return (
    <div className="flex flex-col h-full bg-[#0A0F2A] text-white">

      {/* Messages */}
      <div
        ref={scrollRef}
        className="flex-1 overflow-auto p-4 space-y-4 bg-[#0F1538]"
      >
        {messages.map((msg, i) => (
          <div
            key={i}
            className={`p-3 rounded-lg max-w-[70%] whitespace-pre-wrap ${
              msg.role === 'user'
                ? 'bg-blue-600/20 self-end'
                : 'bg-white/10 self-start'
            }`}
          >
            {msg.content}
          </div>
```

```
      ))}
    </div>

    {/* Input */}
    <div className="p-4 border-t border-white/10 flex gap-2">
      <input
        className="flex-1 p-3 rounded-lg bg-[#0F1538] border border-white/10
                focus:border-blue-500/40 focus:outline-none"
        placeholder="Type a message… or /ai ask something"
        value={input}
        onChange={(e) => setInput(e.target.value)}
        onKeyDown={(e) => e.key === 'Enter' && sendMessage()}
      />

      <button
        onClick={sendMessage}
        className="px-4 py-2 rounded-lg bg-blue-600 hover:bg-blue-500 transition"
      >
        Send
      </button>
    </div>
  </div>
 )
}
```

🧩 What this gives you

✓ AI messages appear directly in the chat stream
They look like another participant.
✓ /ai command support
Users can type:

```
/ai summarize this thread
/ai write a reply
/ai debug this code
```

✓ Full OS-level AI integration
Chat now uses the same AI kernel as the AIPanel.
✓ Auto-scrolling
Feels like a real messaging app.
✓ Zero business logic in the UI
All intelligence stays inside AIProvider and RabbitAIClient.

🎨 Step 2 — Add an “Ask rabbit.ai” button
Inside the ChatWindow input area, add:

```
<button
  onClick={() => {
    if (!input.trim()) return
    run({ prompt: input }).then((result) => {
```

```
    setMessages((m) => [
      ...m,
      { role: 'assistant', content: result }
    ])
  })
  setInput('')
 }}
 className="px-4 py-2 rounded-lg bg-purple-600 hover:bg-purple-500 transition"
>
 Ask rabbit.ai
</button>
```

1. Create the new HopHub module structure
Inside src/modules/, create:

```
hophub/
  ├── HubModule.tsx
  ├── sidebar/
  │     └── HubSidebar.tsx
  ├── channels/
  │     ├── TextChannel.tsx
  │     ├── VoiceChannel.tsx
  │     ├── MeetingChannel.tsx
  │     └── SpaceChannel.tsx   ← HopSpaces lives here
  └── spaces/
        ├── GalaxyView.tsx
        ├── SpaceInterior.tsx
        └── useHopSpaces.ts
```

This is the new home for HopSpaces.
🪐 **2. Move HopSpaces files into the new location**
Move:

src/modules/spaces/GalaxyView.tsx src/modules/spaces/SpaceInterior.tsx
src/modules/spaces/useHopSpaces.ts

To:

src/modules/hophub/spaces/
Delete the old spaces/ module folder afterward.

🧩 **3. Create SpaceChannel.tsx (the new entrypoint)**
This is the component that opens HopSpaces inside HopHub.
src/modules/hophub/channels/SpaceChannel.tsx:

```
import React from 'react'
import GalaxyView from '../spaces/GalaxyView'
import SpaceInterior from '../spaces/SpaceInterior'
```

```
import { useHopSpaces } from '../spaces/useHopSpaces'

export default function SpaceChannel() {
  const { activeSpace } = useHopSpaces()

  return (
    <div className="flex flex-col h-full">
      {activeSpace ? <SpaceInterior /> : <GalaxyView />}
    </div>
  )
}
```

This gives you:

- GalaxyView when no space is selected
- SpaceInterior when a space is active

Exactly how HopSpaces is meant to behave.

🧭 **4. Update HopHub's sidebar to include Space Channels**

src/modules/hophub/sidebar/HubSidebar.tsx:

```
import React from 'react'

export default function HubSidebar({ channels, onSelect }) {
  return (
    <div className="w-64 bg-[#0F1538] text-white p-4 space-y-2">
      {channels.map((ch) => (
        <button
          key={ch.id}
          onClick={() => onSelect(ch)}
          className="w-full text-left px-3 py-2 rounded hover:bg-white/10"
        >
          {ch.type === 'space' ? `🪐 ${ch.name}` : ch.name}
        </button>
      ))}
    
  )
}
```

Space channels now appear with a little planet icon.

🖥️ **5. Update HubModule to route channels**

src/modules/hophub/HubModule.tsx:

```
import React, { useState } from 'react'
import HubSidebar from './sidebar/HubSidebar'
import TextChannel from './channels/TextChannel'
import VoiceChannel from './channels/VoiceChannel'
import MeetingChannel from './channels/MeetingChannel'
import SpaceChannel from './channels/SpaceChannel'

export default function HubModule() {
  const [activeChannel, setActiveChannel] = useState(null)
```

```
 function renderChannel() {
  if (!activeChannel) return null

  switch (activeChannel.type) {
    case 'text': return <TextChannel />
    case 'voice': return <VoiceChannel />
    case 'meeting': return <MeetingChannel />
    case 'space': return <SpaceChannel />
    default: return null
  }
 }

 return (
  <div className="flex h-full">
    <HubSidebar
      channels={[
        { id: 1, type: 'text', name: 'general' },
        { id: 2, type: 'meeting', name: 'Meeting Room' },
        { id: 3, type: 'space', name: 'The Observatory' },
        { id: 4, type: 'space', name: 'The Garden' }
      ]}
      onSelect={setActiveChannel}
    />

    <div className="flex-1 bg-[#0A0F2A]">
      {renderChannel()}
    </div>
  </div>
 )
}
```

HopSpaces is now fully integrated into HopHub.

🚀 **6. Update the Router to use HopHub instead of SpacesModule**

In Router.tsx, replace:

```
} />
```

with:

```
} />
```

SpacesModule is no longer needed.

🎉 **HopSpaces is now officially a sub-menu inside HopHub**

You now have:

✓ HopSpaces as a channel type
✓ GalaxyView + SpaceInterior inside HopHub
✓ Clean routing
✓ Clean UI separation
✓ OS-level modularity
✓ Future-proof structure for AI-powered world building

Goal: rabbit.ai becomes the "World Builder" inside HopSpaces
You'll get:

- AI-powered **Space creation**
- AI-powered **Space editing**
- AI-powered **Galaxy generation**
- AI-powered **entity creation** (objects, rooms, moods)
- AI-powered **metadata updates**
- AI-powered **Space descriptions + lore**

All using your existing useAI() kernel hook.
Let's do this step by step.

🧠 1. Add AI hooks to useHopSpaces
Open:

src/modules/hophub/spaces/useHopSpaces.ts

Add AI integration:

```
import { useAI } from '../../../kernel/providers/AIProvider'

export function useHopSpaces() {
  const { run } = useAI()

  async function createSpaceFromAI(prompt) {
    const result = await run({
      prompt: `Create a new HopSpace with this description: ${prompt}`
    })

    // result should be JSON describing the space
    const space = JSON.parse(result)

    // TODO: save to Supabase or local state
    return space
  }

  async function updateSpaceFromAI(spaceId, prompt) {
    const result = await run({
      prompt: `Modify HopSpace ${spaceId} with this instruction: ${prompt}`
    })

    const updated = JSON.parse(result)
    return updated
  }

  return {
    // existing exports…
    createSpaceFromAI,
    updateSpaceFromAI
  }
```

```
}
```

This gives HopSpaces **AI-powered creation and editing**.

🎨 2. Add an AI button to GalaxyView

Open:

src/modules/hophub/spaces/GalaxyView.tsx

Add a floating AI action:

```
import { useHopSpaces } from './useHopSpaces'
import { useState } from 'react'

export default function GalaxyView() {
  const { createSpaceFromAI } = useHopSpaces()
  const [input, setInput] = useState('')
  const [loading, setLoading] = useState(false)

  async function generate() {
    setLoading(true)
    const space = await createSpaceFromAI(input)
    setLoading(false)

    console.log('AI created space:', space)
  }

  return (
    <div className="relative h-full">

      {/* Galaxy UI here… */}

      {/* AI Creation Panel */}
      <div className="absolute bottom-6 right-6 bg-[#0F1538] p-4 rounded-lg border border-white/10
w-80">
        <textarea
          className="w-full p-2 rounded bg-black/20 border border-white/10"
          placeholder="Describe a new Space…"
          value={input}
          onChange={(e) => setInput(e.target.value)}
        />

        <button
          onClick={generate}
          disabled={loading}
          className="mt-2 w-full py-2 rounded bg-blue-600 hover:bg-blue-500"
        >
          {loading ? 'Generating…' : 'Create with rabbit.ai'}
        </button>
      </div>
    </div>
  )
```

```
}
```

This gives you:

- A floating AI creation box
- Users describe a Space
- rabbit.ai generates the Space metadata
- Galaxy updates instantly

🪐 3. Add AI editing inside SpaceInterior

Open:

src/modules/hophub/spaces/SpaceInterior.tsx

Add an AI “Modify Space” panel:

```
import { useHopSpaces } from './useHopSpaces'
import { useState } from 'react'

export default function SpaceInterior() {
  const { activeSpace, updateSpaceFromAI } = useHopSpaces()
  const [input, setInput] = useState('')
  const [loading, setLoading] = useState(false)

  async function modify() {
    setLoading(true)
    const updated = await updateSpaceFromAI(activeSpace.id, input)
    setLoading(false)

    console.log('AI updated space:', updated)
  }

  return (
    <div className="relative h-full">

      {/* Space interior UI… */}

      {/* AI Editing Panel */}
      <div className="absolute bottom-6 right-6 bg-[#0F1538] p-4 rounded-lg border border-white/10
w-80">
        <textarea
          className="w-full p-2 rounded bg-black/20 border border-white/10"
          placeholder="Modify this Space…"
          value={input}
          onChange={(e) => setInput(e.target.value)}
        />

        <button
          onClick={modify}
          disabled={loading}
          className="mt-2 w-full py-2 rounded bg-purple-600 hover:bg-purple-500"
        >
          {loading ? 'Updating…' : 'Update with rabbit.ai'}
        </button>
```

```
    </div>
   </div>
  )
}
```

Now rabbit.ai can:

- change the mood
- change the background
- add entities
- rewrite the description
- adjust the layout
- evolve the Space

All from inside the Space itself.

THE GALAXY EVOLUTION ENGINE (GEE)

A clean, OS-level subsystem that controls:

1. Mood-driven color shifts
2. Time-based galaxy cycles
3. Activity-based star density changes
4. AI-generated evolution events
5. Space-specific micro-evolutions
6. Global galaxy "seasons"
7. Ambient motion + parallax changes

This engine lives in one place:

src/modules/hophub/spaces/galaxy-engine/

Let's build it step by step.

🧠 **1. Create the Galaxy Engine Hook**

Create:

src/modules/hophub/spaces/galaxy-engine/useGalaxyEngine.ts

```
import { useState, useEffect } from 'react'
import { useAI } from '../../../../kernel/providers/AIProvider'

export function useGalaxyEngine() {
  const { run } = useAI()

  const [mood, setMood] = useState('calm')
  const [starDensity, setStarDensity] = useState(0.4)
  const [nebulaColor, setNebulaColor] = useState('#3A4BFF')
  const [cycle, setCycle] = useState('dawn') // dawn, day, dusk, night

  // Time-based cycle evolution
  useEffect(() => {
    const interval = setInterval(() => {
      setCycle((prev) => {
        if (prev === 'dawn') return 'day'
        if (prev === 'day') return 'dusk'
        if (prev === 'dusk') return 'night'
        return 'dawn'
```

```
    })
  }, 1000 * 60 * 5) // every 5 minutes

  return () => clearInterval(interval)
}, [])

// AI-driven evolution event
async function evolveGalaxy(context) {
  const result = await run({
    prompt: `
      Based on this context, generate a galaxy evolution event:
      ${JSON.stringify(context)}
    `
  })

  const evolution = JSON.parse(result)

  if (evolution.mood) setMood(evolution.mood)
  if (evolution.starDensity) setStarDensity(evolution.starDensity)
  if (evolution.nebulaColor) setNebulaColor(evolution.nebulaColor)
}

return {
  mood,
  starDensity,
  nebulaColor,
  cycle,
  evolveGalaxy
}
}
```

This gives you:

- **time-based evolution**
- **AI-driven evolution**
- **mood + density + color state**
- **a single engine hook for the entire galaxy**

🎨 **2. Connect the Engine to GalaxyBackground**

Open:

src/modules/hophub/spaces/GalaxyView.tsx

Add:

```
import { useGalaxyEngine } from './galaxy-engine/useGalaxyEngine'

export default function GalaxyView() {
  const { mood, starDensity, nebulaColor, cycle, evolveGalaxy } = useGalaxyEngine()

  // Trigger AI evolution every time user enters the galaxy
  useEffect(() => {
    evolveGalaxy({
```

```
    userActivity: 'entered_galaxy',
    currentCycle: cycle,
    currentMood: mood
  })
}, [])

return (
  <div className="relative h-full">
    <GalaxyBackground
      mood={mood}
      starDensity={starDensity}
      nebulaColor={nebulaColor}
      cycle={cycle}
    />

    {/* rest of your GalaxyView UI */}
  </div>
)
}
```

Now the galaxy:

- **reacts to mood**
- **reacts to time**
- **reacts to AI events**
- **reacts to user activity**

🌈 **3. Update GalaxyBackground to respond to evolution**

Open:

src/modules/hophub/spaces/GalaxyBackground.tsx

Add props:

```
export default function GalaxyBackground({ mood, starDensity, nebulaColor, cycle }) {
  const cycleOverlay = {
    dawn: 'rgba(255, 200, 150, 0.15)',
    day: 'rgba(255, 255, 255, 0.05)',
    dusk: 'rgba(255, 100, 150, 0.15)',
    night: 'rgba(0, 0, 50, 0.3)'
  }[cycle]

  return (
    <div className="absolute inset-0 overflow-hidden">
      <div
        className="absolute inset-0 transition-all duration-1000"
        style={{
          background: `radial-gradient(circle at 50% 50%, ${nebulaColor}, #000)`,
          opacity: mood === 'calm' ? 0.8 : mood === 'storm' ? 1 : 0.9
        }}
      />

      <Starfield density={starDensity} />
```

```
    <div
      className="absolute inset-0 pointer-events-none transition-all duration-1000"
      style={{ background: cycleOverlay }}
    />
  </div>
 )
}
```

Now the galaxy:

- shifts color
- shifts density
- shifts mood
- shifts cycle
- animates smoothly

🪐 **4. Add AI-Driven "Galaxy Seasons"**

Inside useGalaxyEngine:

```
useEffect(() => {
 const interval = setInterval(() => {
  evolveGalaxy({
    event: 'season_tick',
    cycle,
    mood
  })
 }, 1000 * 60 * 30) // every 30 minutes

 return () => clearInterval(interval)
}, [cycle, mood])
```

Now the galaxy evolves:

- every 30 minutes
- based on AI
- based on mood
- based on cycle

This is how you get:

- **Aurora Season**
- **Storm Season**
- **Bloom Season**
- **Void Season**

All generated by rabbit.ai.